

**Universidade Federal de Pernambuco
Centro de Informática - CIn**

**SAMP - SISTEMA DE ACOMPANHAMENTO
DE INFORMAÇÃO DE MERCADO PARA REGULARIZAÇÃO ECONÔMICA**



Relatório do Projeto de Integração
Banco de dados - 2026.1

Anysabele de Paula Barbosa Santos - <apbs2>
Caio Ferreira Gomes da Silva - <cfgs>
Dyego Ferreira da Silva - <dfs10>
Jairo Cândido Gonzaga Neto - <jcgn>
Júlio César da Silva - <jcs8>
Vinicius Guedes de Macedo - <vgm>

1. Descrição da Base de Dados Unificada.

A base unificada consiste em um Data Warehouse modelado em Esquema Estrela (Star Schema), focado no acompanhamento do mercado de energia elétrica (SAMP/ANEEL). A modelagem centraliza as métricas financeiras e de consumo na tabela **fato_energia** (contendo o Valor do Mercado e chaves estrangeiras), rodeada por cinco dimensões descritivas:

- **dim_tempo**: Granularidade mensal e trimestral baseada na data de competência.
- **dim_distribuidora**: Identificação (CNPJ, Sigla, Nome) da concessionária.
- **dim_acessante**: Entidades vinculadas à rede, com IDs gerados sequencialmente.
- **dim_mercado**: Categorização tarifária, classe de consumo e posto tarifário.
- **dim_origem**: Metadados de rastreabilidade (data de geração e arquivo CSV de origem).

2. Explicação Detalhada do Processo de Integração.

O processo de integração foi estruturado em três camadas lógicas para garantir a governança e a rastreabilidade dos dados:

- **Ingestão (Camada Bronze)**: Os arquivos brutos (**.csv**) referentes aos anos de 2024, 2025 e 2026 foram extraídos e carregados inicialmente em um ambiente de **staging**. Nesta etapa, nenhuma transformação de regra de negócio foi aplicada; o foco foi garantir a integridade da carga dos dados originais conforme disponibilizados pela ANEEL.
- **Processamento e Limpeza (Camada Silver)**: Os dados brutos foram submetidos a processos de higienização automatizados. Nesta fase, corrigimos inconsistências textuais, padronizamos esquemas de codificação e tipagem de colunas de datas. O objetivo aqui foi transformar o dado bruto "sujo" em uma base limpa e pronta para modelagem.
- **Modelagem Dimensional (Camada Gold)**: A etapa final consistiu na decomposição dos *DataFrames* limpos em uma arquitetura de Esquema Estrela (Star Schema). Identificamos as métricas quantitativas para a **Tabela Fato (fato_energia)** e isolamos os atributos descritivos em **Tabelas Dimensão (dim_tempo, dim_distribuidora, dim_mercado, dim_acessante e dim_origem)**. Este modelo consolidado foi então injetado no SGBD PostgreSQL, garantindo alta performance para consultas analíticas.

3. Justificativa da Escolha da Base de Dados.

A base de dados escolhida foi o SAMP - Sistema de Acompanhamento de Informações de Mercado para Regulação Econômica, disponibilizado pela **ANEEL**. A seleção foi motivada pela relevância do setor elétrico no contexto nacional e pela importância estratégica das informações de geração e distribuição de energia para análises de mercado e tomada de decisão.

Além disso, a base é pública, confiável e apresenta boa estruturação dos dados, com variáveis temporais e operacionais que permitem análises consistentes ao longo do tempo. Essas características tornam o dataset adequado para a aplicação de técnicas de engenharia de dados, especialmente modelagem dimensional em esquema estrela e construção de pipelines ETL/ELT.

Por fim, o SAMP oferece um ambiente apropriado para exploração analítica e simulação de cenários reais, contribuindo diretamente para o desenvolvimento de competências em integração e transformação de dados.

4. Descrição dos Processos de Transformação Aplicados.

4.1. Transformações via ETL (Extract-Transform-Load)

Nesta abordagem, a inteligência e o esforço computacional residem **fora** do banco de dados (no Python/Pandas). Os dados chegam ao banco de dados já "impecáveis" e modelados.

- **Extração:** Leitura dos arquivos brutos `.csv` via `pandas.read_csv()`.
- **Transformação (Executada no ambiente do VS Code):**
 - **Higienização de Strings:** Uso de `unicodedata` e `str.upper()` para padronizar textos e remover acentos.
 - **Casting de Tipos:** Conversão de formatos de data (usando `parse_dates`) e de valores numéricos para garantir que o banco de dados receba o tipo correto de dado.
 - **Modelagem Dimensional:** Criação de *DataFrames* separados para cada Dimensão (`dim_tempo`, `dim_distribuidora`, etc.) e para a Tabela Fato antes de qualquer contato com o banco.
 - **Geração de Chaves:** Criação de chaves artificiais (Surrogate Keys) via lógica de dicionários ou sequenciais do Python.

Carga: Envio final dos *DataFrames* limpos para o PostgreSQL através do `to_sql` do SQLAlchemy.

4.2. Transformações via ETL (Extract-Transform-Load)

Aqui, o banco de dados é o protagonista. O Python serve apenas como um "tubo" para transportar os dados brutos, e a inteligência do processamento é transferida para o **SQL**.

- **Extração:** Leitura dos arquivos brutos via Python.
- **Carga (Staging):** Injeção de todos os dados "sujeitos" em uma única tabela temporária chamada **Dados** ou `stg_samp` dentro do PostgreSQL, sem qualquer alteração prévia
- **Transformação (Executada dentro do PostgreSQL via SQL ou dbt):**
 - **Criação da Estrutura:** Execução de scripts SQL (`CREATE TABLE AS SELECT ...`) para separar a tabela bruta nos modelos Fato e Dimensões.
 - **Limpeza (SQL):** Uso de funções como `UPPER()`, `TRIM()`, `REPLACE()` e `CAST()` dentro das queries SQL para limpar strings e converter tipos, substituindo as funções do Pandas.
 - **Validação:** Execução de testes de integridade (como checagem de nulos ou valores únicos) via **dbt** ou comandos SQL **ASSERT** após a carga.

5. Comparativo entre ETL e ELT.

A implementação das duas arquiteturas (ETL e ELT) neste projeto evidenciou diferenças significativas quanto à flexibilidade, uso de recursos e escalabilidade.

5.1. Abordagem ETL (Extração, Transformação e Carga via Python/Pandas):

- **Vantagens:** O uso do Python, especificamente com a biblioteca Pandas, proporcionou um controle extremamente granular sobre os dados. Transformações complexas, como a remoção de acentuação utilizando a biblioteca **unicodedata** e o tratamento rigoroso de *strings* nulas ou mal formatadas, foram implementadas de forma limpa e programática. Além disso, a geração de chaves artificiais numéricas (Surrogate Keys) em memória para a dimensão de acessantes foi executada com facilidade.
- **Desvantagens:** Como a transformação ocorre em memória RAM antes da inserção no banco de dados, essa abordagem possui um gargalo de hardware. O processamento integral dos arquivos de 2024 a 2026 exigiu um gerenciamento cuidadoso da memória.
- **Adequação ao Projeto:** Demonstrou-se a arquitetura mais eficiente e resolutiva para a nossa entrega, pois a complexidade da limpeza textual dos dados da ANEEL foi facilmente superada pelas funções nativas do Python.

5.2. Abordagem ELT (Extração, Carga e Transformação via SQL/Banco de Dados):

- **Vantagens:** O ELT brilhou na etapa de ingestão. Carregar os dados brutos (Camada *Bronze*) diretamente no PostgreSQL foi um processo rápido e que evitou o estrangulamento da memória local. Toda a transformação (Camada *Silver* e *Gold*) passa a ser responsabilidade do motor otimizado do banco de dados, o que confere altíssima escalabilidade estrutural.
- **Desvantagens:** Expressar regras de limpeza de *strings* complexas (como remoção de múltiplos espaços internos ou mapeamento de caracteres especiais) puramente em SQL exige *queries* muito mais verbosas e difíceis de dar manutenção em comparação ao Python.
- **Adequação ao Projeto:** Embora ideal para ambientes em nuvem com alta capacidade de processamento, no contexto deste projeto acadêmico, a orquestração completa das transformações intrincadas via SQL demandou uma curva de configuração mais longa que o script direto em Pandas.

5.3 Quadro Comparativo:

Característica	ETL (Python/Pandas)	ELT (SQL/Postgres)
Onde ocorre a transformação?	Fora do banco (no Python)	Dentro do banco (via SQL)
Complexidade de limpeza	Alta (fácil com bibliotecas)	Média (exige queries mais longas)
Performance de Carga	Mais lenta (espera o processamento)	Muito rápida (cargas brutas)
Escalabilidade	Limitada pela RAM da máquina	Alta (usa o poder do servidor DB)
Ferramentas principais	Pandas, NumPy	SQL, dbt, Stored Procedures

6. Apresentação de Três Análises e Insights.

Com o Esquema Estrela estruturado e os dados higienizados no PostgreSQL, foi possível realizar análises complexas sobre o mercado de energia elétrica brasileiro.

6.1. Evolução Temporal da Receita (Sazonalidade Financeira):

- **A Análise:** Cruzamos a **fato_energia** com a **dim_tempo** filtrando pela métrica "RECEITA ENERGIA (R\$)" para agrupar o faturamento por ano e trimestre.
- **Insight Obtido:** A consulta SQL permitiu visualizar padrões de sazonalidade no faturamento das concessionárias. Picos de receita são comumente observados no primeiro e último trimestres do ano (Q1 e Q4), o que está diretamente atrelado ao aumento de consumo físico gerado pelas altas temperaturas do verão e pelo uso intensivo de equipamentos de refrigeração.

6.2. Concentração de Mercado (Logística de Distribuição):

- **A Análise:** Realizamos um ranqueamento das maiores empresas do país agrupando o "ENERGIA TE (KWH)" distribuído através das informações da **dim_distribuidora**.
- **Insight Obtido:** A análise do Market Share logístico revelou que o mercado nacional é altamente concentrado. Uma pequena parcela de grandes distribuidoras é responsável pelo escoamento da esmagadora maioria dos Megawatts/Kilowatts do país, enquanto dezenas de outras operam em fatias regionais mínimas.

6.3. Cobertura de Grupos por Classe de Consumo:

- **A Análise:** Agrupamos o volume financeiro faturado com base na descrição contida na coluna de Classe de Consumo da **dim_mercado** (Residencial, Industrial, Comercial, etc.).
- **Insight Obtido:** Esta análise atua como um termômetro macroeconômico. O cruzamento demonstrou quais classes representam a maior base pagadora do sistema elétrico. O monitoramento contínuo destas fatias permite, por exemplo, identificar se houve retração na classe "Industrial", o que sinalizaria uma desaceleração na produção e na economia do país.

7. Reflexão sobre o Aprendizado.

O desenvolvimento completo deste pipeline de dados representou um salto qualitativo no entendimento da arquitetura de banco de dados e da governança da informação. O maior desafio superado foi lidar com o estado real de bases de dados públicas. Os dados originais da ANEEL apresentavam tipagens inconsistentes e erros descritivos (como falta de acentuação e espaços duplos não intencionais) que geram duplicidades lógicas críticas.

O processo de estruturar lógicas de limpeza, converter essas informações em Surrogate Keys confiáveis e desenhar um Esquema Estrela (Fato e Dimensões) funcional nos ensinou que, na Engenharia de Dados, a higienização prévia corresponde à maior parte do esforço técnico. Entender essas dinâmicas de carga, normalização e consultas otimizadas fornece uma bagagem analítica indispensável, fundamentando uma base extremamente competitiva para futuras atuações práticas e superação de testes técnicos de desenvolvimento de software em grandes instituições financeiras, como Itaú, Bradesco ou Santander, onde a manipulação robusta de alto volume de dados transacionais via SQL é avaliada com alto rigor.